Web Development through React

Prepared by: Godwin Lorenz B. Llabres | Instructor I
John Aimiel C. Pena | Instructor I

College of Engineering and Information Technology

What is React?

Introduction to React

What is React?

It's used for building web and mobile applications by breaking them down into reusable, modular components, making development faster, more maintainable, and more scalable. Key reasons to use React include its component-based architecture, efficient rendering with a Virtual DOM, reusability, declarative syntax, and a large, active community and ecosystem.

Introduction to React

What React is

- **A JavaScript Library:** Unlike a full framework, React provides tools and APIs for UI development but leaves other aspects of application building (like routing or state management) to the developer or third-party libraries.
- **Component-Based**: React promotes a modular approach by allowing developers to create self-contained, reusable UI building blocks called components.
- **Declarative**: You describe what the UI should look like based on the application's current state, and React handles the updates to the actual user interface efficiently.
- **Uses JSX**: React uses JSX, a syntax extension for JavaScript, to write UI with an HTML-like structure within JavaScript code.

Introduction to React

Why React

- **Scalability**: Its modular, component-based architecture simplifies the creation of large and complex applications, making them easier to manage and update as they grow.
- **Performance**: React's Virtual DOM allows it to efficiently update only the necessary parts of the UI when changes occur, rather than re-rendering the entire page, leading to faster and more responsive applications.
- **Reusability**: Components can be reused across different parts of an application or even in other projects, saving development time and effort and ensuring consistency.
- **Developer Experience**: React's declarative nature and component-based approach lead to more maintainable and testable code.
- **Active Ecosystem & Community**: React has a huge community and a vast ecosystem of third-party libraries and tools, providing robust solutions for various development needs.
- **SEO-Friendly**: React's ability to perform server-side rendering helps search engines to effectively index and rank web pages, improving SEO performance.

Introduction to React

Core Concept: Components

React apps are built with components

What are components?

- reusable building blocks of UI (User interface)

Key Characteristics of Components:

1. Reusable
2. Independent
3. Encapsulate UI and Logic
4. Accept inputs (props) – properties
5. Manage states
6. Return JSX

Types of Components:

1. Functional Components
2. Class Components

Introduction to React

Core Concept: Components

Functional Components

```
import React from 'react';

function Greeting(props) {
      return (
                <h1>Hello, {props.name}!</h1>;
      )
   }
export default Greeting;
```

Class Components

```
 import React, { Component } from 'react';

class Welcome extends Component {              render() {
                        return <h1>Welcome, {this.props.name}!</h1>;
    }
   }
   export default Welcome;
```

Introduction to React

Props

Props or Properties

Purpose: Props are used to pass data from a parent component to a child component, similar to how arguments are passed to a function. They enable customization and configuration of child components by the parent.

Mutability: Props are immutable within the child component that receives them. A child component should treat its props as read-only and not attempt to modify them directly. The parent component is the "source of truth" for the props it passes down.

```jsx
// ParentComponent.jsx
import React from 'react';
import Button from './Button';

function ParentComponent() {
  return <Button color="blue" text="Click Me" />;
}

export default ParentComponent;

// Button.jsx
import React from 'react';

function Button(props) {
  return <button style={{ backgroundColor: props.color }}>{props.text}</button>;
}

export default Button;
```

Introduction to React

State

Purpose: State is used to manage internal, mutable data within a component. It allows a component to keep track of information that can change over time due to user interactions, API responses, or other internal logic.

Mutability: State is local to the component it belongs to and can be modified only by that component using setState in class components or the useState hook in functional components. Any change in state triggers a re-render of the component.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
    </div>
  );
}

export default Counter;
```

What is React?

Component-Based Architecture

React applications are built using components - reusable, self-contained pieces of code that represent parts of a user interface. Components follow the Single Responsibility Principle and encapsulate both UI logic and presentation.

Key Points:

1. Components are pure functions that transform props and state into React elements (Virtual DOM nodes)
2. Functional components use hooks for state and lifecycle management, while class components use lifecycle methods
3. Component composition creates hierarchical UI trees with unidirectional data flow
4. Each component has its own scope, state management, and can implement custom lifecycle behaviors
5. Components support TypeScript for type safety and better developer experience
6. Higher-Order Components (HOCs) and Render Props patterns enable advanced composition strategies

What is React?

Declarative Programming Paradigm

React embraces declarative programming where you describe the desired UI state rather than imperative DOM manipulation steps. This paradigm shift enables predictable, maintainable code through functional programming principles.

1. Declarative code describes 'what' the UI should look like for any given state, not 'how' to achieve it
2. React's reconciliation algorithm efficiently updates the DOM using a Virtual DOM diffing process
3. Functional programming concepts like immutability and pure functions reduce side effects
4. State transitions are predictable and can be reasoned about mathematically
5. Time-travel debugging and state replay become possible with predictable state changes
6. Declarative patterns enable better testing through predictable input/output relationships

What is React?

State-Driven UI Architecture

React implements unidirectional data flow where UI is a pure function of state. State management drives all UI updates through a predictable, reactive system that ensures UI consistency and enables powerful debugging capabilities.

1. State represents the complete UI condition at any point in time, following the single source of truth principle
2. React's reconciliation engine automatically re-renders components when state changes, using efficient diffing
3. Unidirectional data flow prevents circular dependencies and makes state changes predictable
4. State should be treated as immutable to enable time-travel debugging and performance optimizations
5. Hook-based state management (useState, useReducer) provides fine-grained reactivity
6. State lifting and context provide solutions for sharing state across component boundaries
7. Advanced patterns include state machines, reducers, and reactive programming with observables

What is React?

JavaScript Library Architecture

React is a focused JavaScript library rather than a full framework, implementing the Unix philosophy of 'do one thing and do it well'. This architectural decision provides maximum flexibility while maintaining a minimal core that can be extended with ecosystem tools.

1. Library vs Framework: React provides view layer functionality without imposing architectural constraints
2. Minimal core with extensible plugin architecture through the React ecosystem
3. Can be integrated incrementally into existing applications without complete rewrites
4. Ecosystem flexibility allows choosing best-in-class solutions for routing, state management, and tooling
5. React's core focuses on component rendering, state management, and Virtual DOM reconciliation
6. Framework-like capabilities achieved through composition of libraries (React Router, Redux, etc.)
7. Build tool agnostic - works with Webpack, Vite, Parcel, or even without build tools

What is React?

Industry Adoption & Ecosystem

React has achieved unprecedented adoption in the JavaScript ecosystem, becoming the de facto standard for modern frontend development. Its popularity stems from strong technical foundations, active community, and enterprise-grade tooling ecosystem.

1. Used by Fortune 500 companies including Meta, Netflix, Airbnb, Uber, WhatsApp, and Instagram
2. GitHub: 220,000+ stars, 45,000+ forks, representing massive community engagement
3. NPM ecosystem: 20M+ weekly downloads with 100,000+ React-related packages
4. Developer surveys consistently rank React as most loved and wanted frontend technology
5. Job market: React skills command premium salaries and have highest demand in frontend development
6. Corporate backing from Meta ensures long-term stability and continued innovation
7. Active RFC (Request for Comments) process allows community input on future development

What is React?

Meta Engineering & Open Source

React was created by Jordan Walke at Facebook (now Meta) in 2011 to solve complex UI problems at scale. Its open-source success demonstrates how enterprise solutions can benefit the entire developer community.

1. Originally developed to solve Facebook's news feed performance and complexity issues
2. First production deployment in Facebook's news feed (2011), later Instagram (2012)
3. Open-sourced at JSConf US in May 2013, revolutionizing frontend development
4. Maintained by Meta's dedicated React team with contributions from 1,500+ community contributors
5. React's development philosophy: 'Move Fast and Don't Break Things' through careful API design
6. Concurrent development of React Native (2015) proved React's architecture could transcend web development
7. RFC (Request for Comments) process ensures community input on major changes